LR(0) / SLR(1) Parser

CS3510 — Compiler Construction · Semester Project Report

Course	CS3510 – Compiler Construction
Project	Option A: Parsing Phase
Parser	LR(0) + SLR(1) Bottom-Up Parser
Platform	Web Application (HTML / CSS / JavaScript)
Submitted	Microsoft Teams

GitHub Repository: https://github.com/<your-username>/lr-parser

1. Problem Description

Parsing is the second major phase of a compiler, responsible for determining whether a given input string belongs to the language defined by a context-free grammar (CFG). The parser reads a stream of tokens produced by the lexical analyser and checks syntactic correctness according to the grammar rules. For large, real-world programming languages, bottom-up parsers are preferred because they handle a wider class of grammars than top-down parsers.

This project implements a **bottom-up LR(0) / SLR(1) parser** as a fully interactive web application. The tool allows users to input any context-free grammar (manually or via file), build the canonical LR(0) item sets, construct the SLR(1) parse table, and perform step-by-step parsing of an input string — displaying the parse tree for accepted strings.

2. Selected Compiler Phase

Phase: Parsing (Syntax Analysis)

Parser Type: Bottom-Up — LR(0) and SLR(1)

LR parsers are left-to-right, rightmost-derivation parsers. They are more powerful than LL parsers and are used in most production compilers (GCC, Clang). The key difference between LR(0) and SLR(1) is the lookahead used to resolve reduce conflicts:

Property	LR(0)	SLR(1)
Lookahead	None	FOLLOW set (1 symbol)
Conflicts	More common	Fewer — resolved via FOLLOW
Grammar class	Strict LR(0)	SLR(1) superset of LR(0)
Table construction	Same items	Same items, smarter reduces

3. Design and Implementation Overview

3.1 Architecture

The application is a single-page web app built with vanilla HTML, CSS, and JavaScript — no external frameworks or dependencies required. All parsing logic runs client-side in the browser. The codebase is divided into clearly separated modules:

Module	Responsibility
Grammar Parser	Tokenises CFG text, validates syntax, extracts productions/terminals/NTs
Grammar Augmentor	Adds augmented start S' → S for LR construction
FIRST / FOLLOW	Fixed-point algorithm to compute FIRST and FOLLOW sets for all NTs
LR(0) Builder	Closure and GOTO functions; canonical collection of LR(0) item sets
Table Builder	ACTION (shift/reduce/accept) and GOTO table; conflict detection
Parse Engine	Stack-based LR parse loop; produces step trace and AST nodes
Tree Renderer	Canvas-based recursive parse tree drawing
GUI Layer	Tabbed interface, live status, step table, item set grid, parse table

3.2 Key Algorithms

Closure(I):

For every item [A → α • B β] in the set, add all items [B → • γ] for each B-production. Repeat until no new items are added (fixed-point).

GOTO(I, X):

Move the dot past symbol X in all items where X is the next symbol, then compute the closure of the resulting set.

SLR(1) Table Fill:

Shift entries come from GOTO transitions on terminals. Reduce entries [A → α •] are placed in ACTION[i, a] for all a ∈ FOLLOW(A). The accept action is placed at the augmented start rule. Conflict detection checks for overwriting an existing entry.

3.3 GUI Features

• Three input modes: manual text, file upload (.txt), and built-in examples

• Toggle between LR(0) and SLR(1) parser types

• Six information tabs: Grammar · LR(0) Item Sets · FIRST/FOLLOW · Parse Table · Parse Steps · Parse Tree

• Colour-coded parse table (shift=blue, reduce=red, accept=green, goto=purple)

• Conflict highlighting with warning banner

• Interactive step-by-step trace table showing stack, remaining input, and action

• Canvas-rendered parse tree for accepted strings

4. Sample Inputs and Manual Working

4.1 Example Grammar — Arithmetic Expressions

```
E -> E + T | T
T -> T * F | F
F -> ( E ) | id
```

After augmentation, the grammar gains production **E' → E**. The parser builds 12 LR(0) states for this grammar. The SLR(1) table has no conflicts.

4.2 Parse Trace — Input: `id + id * id`

Step	Stack	Input Remaining	Action
1	0	id + id * id $	Shift → s5
2	0 5	+ id * id $	Reduce r6: F → id
3	0 3	+ id * id $	Reduce r4: T → F
4	0 2	+ id * id $	Reduce r2: E → T
5	0 1	+ id * id $	Shift → s6
6	0 1 6	id * id $	Shift → s5
7	0 1 6 5	* id $	Reduce r6: F → id
8	0 1 6 3	* id $	Shift → s7
9	0 1 6 3 7	id $	Shift → s5
10	0 1 6 3 7 5	$	Reduce r6: F → id
11	0 1 6 3 7 10	$	Reduce r5: T → T*F
12	0 1 6 9	$	Reduce r1: E → E+T
13	0 1	$	ACCEPT ✓

The string **id + id * id** is accepted after 13 steps. The parser correctly handles operator precedence — multiplication binds tighter than addition — due to the grammar structure.

5. Results and Observations

5.1 Test Cases

#	Grammar	Input	Parser	Result
1	Arithmetic Expr	id + id * id	SLR(1)	ACCEPTED ✓
2	Arithmetic Expr	(id + id)	SLR(1)	ACCEPTED ✓
3	Arithmetic Expr	id + * id	SLR(1)	REJECTED ✗
4	Simple S→aAb	a c b	LR(0)	ACCEPTED ✓
5	Simple S→aAb	a a b	LR(0)	REJECTED ✗
6	Balanced Parens	(( ))	SLR(1)	ACCEPTED ✓

5.2 GUI Screenshot Descriptions

Grammar Tab: Displays the augmented grammar with production numbers, terminals, non-terminals, and start symbol pills.

LR(0) Items Tab: Shows all canonical LR(0) item sets as cards with the dot position highlighted and GOTO transitions listed.

FIRST/FOLLOW Tab: Side-by-side grid of FIRST and FOLLOW sets for every non-terminal.

Parse Table Tab: Colour-coded ACTION/GOTO table. Conflict cells are highlighted in red with a warning banner listing all conflicts.

Parse Steps Tab: Numbered trace table: stack state numbers, remaining input, and action taken at each step. Final row is green (accept) or red (reject).

Parse Tree Tab: Canvas-drawn tree for accepted strings. Internal nodes are blue (non-terminals); leaf nodes are green (terminals).

6. Challenges and Observations

Epsilon Productions: Productions with ε required special-casing throughout. The closure algorithm must not advance the dot for ε, and the FIRST computation must propagate ε correctly through sequences.

Conflict Detection: When both a shift and a reduce entry compete for the same cell, the table builder records both and highlights the conflict rather than silently overwriting — giving the user diagnostic information.

LR(0) vs SLR(1) Difference: Grammars that cause conflicts under LR(0) often become conflict-free under SLR(1) because reduces are restricted to tokens in FOLLOW(A) rather than all terminals. The toggle switch lets students observe this difference interactively.

Parse Tree Layout: Automatically laying out a tree on a fixed canvas without overlap is non-trivial. The solution assigns horizontal space proportional to the number of leaves beneath each node, ensuring no two subtrees overlap regardless of grammar depth.

No External Libraries: Keeping the tool dependency-free means it runs offline and can be submitted as a single HTML file — simplifying submission and ensuring reproducibility on any machine with a browser.

All test cases passed. The implementation correctly demonstrates LR(0) item construction, SLR(1) table building, conflict detection, step-by-step parsing, and parse tree generation as required by the CS3510 project specification.